

Yggdrasil Platform — Mimisbrunnr: Canonical Data Architecture and Integration Framework

Mimir Labs Technical Publication

Document Version: 1.0

Date: March 2026

Classification: Public

Overview

Mimisbrunnr is the canonical semantic model at the center of the Mimir Labs data platform. Named for the well of wisdom in Norse mythology, it serves as both the authoritative schema definition for the Yggdrasil ERP system and the universal reference vocabulary used by every tool in the platform stack.

The schema itself contains 166 PostgreSQL tables organized across 17 business domains, covering the operational breadth expected of a modern enterprise system. But the architectural significance of Mimisbrunnr extends beyond any single application. It functions as the Rosetta Stone that enables semantic interoperability between systems, tools, and integration layers across the entire Mimir Labs architecture.

This paper describes Mimisbrunnr in two dimensions: as the governed persistence model that underpins Yggdrasil's operational execution, and as the canonical reference framework consumed by Ratatosk, Ragnarok, Bifrost, and Jormungandr.

The Cardinal Rule

One principle governs all interaction with Mimisbrunnr: the canonical schema is authoritative.

Target tables and columns are never dropped, renamed, or structurally modified to accommodate external systems. Source elements must adapt to Mimisbrunnr, not the reverse. Type incompatibilities are resolved by transforming source data, not by changing target definitions. Missing required fields in source data generate gap reports, not schema modifications.

This constraint is not arbitrary. It exists because semantic stability is the precondition for every downstream process in the platform. If the canonical model shifts to accommodate each new integration, it ceases to function as a reference and becomes another moving target in an already fragmented enterprise landscape.

Architectural Role

Mimisbrunnr occupies a unique position in the Mimir Labs architecture. It is simultaneously a working production schema and a platform-wide semantic reference.

Within Yggdrasil, Mimisbrunnr is the database. Every transaction, every status transition, every audit record is persisted against its structures. The server's state machine, route handlers, and query builders all operate directly on Mimisbrunnr tables.

Outside Yggdrasil, Mimirbrunnr is a vocabulary. Ratatosk uses it to align discovered enterprise entities with canonical definitions. Ragnarok uses it to validate migration targets and generate type-compatible transformation pipelines. Bifrost uses it to route synchronized data between systems using a shared semantic model. Jormungandr uses it to detect governance drift by comparing incoming schemas against canonical expectations.

This dual role is architecturally significant. The same schema that serves as a production database also serves as the semantic foundation for the entire tool ecosystem. That convergence eliminates the common enterprise problem of maintaining separate canonical models and operational schemas that inevitably drift apart.

Schema Organization

The 166 tables are organized into 17 logical domains that span the operational scope of an enterprise system.

Core business domains include customer relationship management, sales operations, purchasing and procurement, manufacturing execution, warehouse and inventory management, financial accounting, project management, product lifecycle management, quality assurance, and service operations. Supporting domains cover human resources, logistics, integration infrastructure, workflow orchestration, form definitions, asset management and MRP planning, and multi-currency accounting.

Although the schema is large, its structure follows consistent patterns that make it predictable and legible. Every entity table uses UUID primary keys, includes tenant isolation columns, carries creation and modification timestamps, and supports soft deletion through nullable timestamp fields. Status fields are implemented as PostgreSQL ENUM types that work in conjunction with the server's state machine to enforce valid lifecycle transitions.

This uniformity is intentional. A schema that serves as both an operational database and a semantic reference must be internally consistent enough that tools can reason about its structure programmatically.

Cross-Domain Relationships

The operational value of Mimirbrunnr lies not in individual tables but in the relationships between domains. These relationships represent real enterprise workflows that span organizational boundaries.

The order-to-cash flow connects customer accounts through quotes, sales orders, invoices, payments, and general ledger entries, while simultaneously triggering work orders, manufacturing operations, pick lists, shipments, and delivery records. The procure-to-pay flow traces from suppliers through purchase orders, receipts, bills, payments, and ledger postings, with parallel inventory transaction and warehouse location updates. The design-to-manufacture flow links PLM parts through engineering and manufacturing bills of materials to work orders, operations, completed units, serial numbers, and asset registries. The issue-to-resolution flow connects service tickets through NCRs, 8D reports, CAPA actions, and audit findings.

These cross-domain chains are not merely data relationships. They represent the operational processes that define how an enterprise functions. By encoding them in the schema, Mimirbrunnr makes enterprise workflows structurally explicit rather than implicitly embedded in application logic.

Integration Framework

Mimisbrunnr includes a dedicated integration subsystem consisting of seven tables that form the backbone of external system connectivity.

The integration endpoint registry stores connection definitions for external systems, including URLs, authentication credentials, and protocol specifications. The message queue manages outbound and inbound payloads as JSONB documents. A delivery log tracks each attempt to deliver a message, recording success or failure status. The dead letter queue preserves messages that have exhausted retry attempts, making them available for investigation and manual resolution. A transformation table stores data mapping rules between Mimisbrunnr fields and external schemas. Webhook listeners enable event-driven callbacks to registered external systems. A synchronization state table tracks last-sync timestamps and cursors for incremental data exchange.

This infrastructure enables three integration patterns. Event-driven push integration delivers real-time notifications to external systems as Yggdrasil events occur. Polling-based pull integration allows external systems to incrementally fetch changed records using timestamp-based delta queries. Batch import enables bulk data loading with idempotency guards and cross-tenant protection.

Event Architecture

The B2B Event Hub provides real-time event streaming over WebSocket channels. Events include state transitions, data changes, workflow activity, and system notifications.

Events are tenant-scoped, authenticated, and emitted from governed application paths rather than from uncontrolled database triggers. The event stream is not an approximation of system activity; it is a structured expression of validated system behavior.

The Redpanda Relay bridges the internal event system to external Kafka-compatible message infrastructure, enabling event archival, cross-deployment synchronization, and downstream analytics pipelines. Topic naming follows a tenant-scoped convention that maintains isolation at the message broker level.

This event architecture is significant because it transforms Mimisbrunnr from a passive persistence layer into an observable operational system. External tools and integration partners can subscribe to the event stream without polling the transactional API, creating a clean separation between operational execution and integration consumption.

Semantic Reference Function

When used as a semantic reference by platform tools, Mimisbrunnr provides several capabilities that go beyond a simple schema definition.

Domain classification establishes contextual boundaries for mapping and validation. The 17-domain taxonomy allows tools like Ratatosk and Ragnarok to scope their operations to relevant business areas, preventing cross-domain ambiguity during schema comparison and data migration.

Type definitions provide a normalization target for heterogeneous source systems. The Type Compatibility layer in Ragnarok, for example, evaluates whether a source column's data type can be safely transformed to match a Mimisbrunnr target column, classifying each mapping as compatible,

lossy, or invalid.

Relationship structures define the dependency graph used for topological sorting during migration. Ragnarok uses Mimisbrunnr's foreign key relationships to determine safe insertion order, ensuring that parent records exist before child records that reference them.

Naming conventions provide the baseline vocabulary for synonym-based semantic matching. The controlled vocabulary of 50 or more synonym groups used by Ratatosk and Ragnarok is rooted in Mimisbrunnr's column and table naming conventions.

Schema Evolution Philosophy

The long-term stability of the platform depends on disciplined schema evolution. Several guiding principles govern this process.

Changes are additive whenever possible. New tables, columns, and ENUM values are appended without modifying existing structures. Structural removals, when necessary, are staged across multiple releases to reduce operational risk. ENUM expansions occur by appending values rather than redefining existing definitions.

Migrations are versioned, sequential, and applied automatically during server startup. Each migration is recorded in a tracking table, and rollback scripts exist for controlled recovery. The migration system treats schema evolution as part of the operational contract of the server, not as an ad hoc maintenance activity.

This discipline ensures that Mimisbrunnr remains stable enough to serve as a canonical reference while evolving to accommodate new operational requirements.

Observability

The integration framework provides visibility into system health through multiple channels.

Message throughput is tracked through the delivery log. Failure rates are visible through the dead letter queue. Endpoint availability is monitored through last-successful-connection timestamps. Event latency is measured from generation to consumer delivery.

Dead letters are accessible through dedicated API endpoints that allow investigation, resolution, and re-queuing of failed messages. Full payload and error history are preserved for each failed delivery.

Design Principles

Six principles govern the Mimisbrunnr architecture.

First, the schema is authoritative. Application code, APIs, and integrations derive from it, not the other way around.

Second, tenant isolation is universal. Every integration message, event, and data record is scoped to a tenant. No shared-state leakage is possible.

Third, delivery semantics are at-least-once. Events may be delivered more than once; consumers must be idempotent.

Fourth, the system fails open for reads and closed for writes. Read failures return empty results; write failures abort the transaction.

Fifth, dead letters are preferred over data loss. Failed messages are preserved for investigation, never silently dropped.

Sixth, schema evolution is additive. New structures are added; existing structures are not modified or removed.

Conclusion

Mimisbrunnr is more than a database schema. It is the semantic foundation upon which the entire Mimir Labs platform operates.

As a production database, it provides governed persistence for Yggdrasil's operational execution. As a canonical reference, it provides the shared vocabulary that enables Ratatosk to discover, Ragnarok to migrate, Bifrost to synchronize, and Jormungandr to govern.

That convergence between operational schema and semantic reference is the architectural insight at the heart of the Mimir Labs platform. By making the production data model and the canonical reference model one and the same, Mimisbrunnr eliminates the drift that typically separates enterprise documentation from enterprise reality.

Copyright 2026 Mimir Labs. All rights reserved.